

Skynet MMO 学习课程第四课：生产工程、故障诊断与性能

面向读者：有多年 MMO Server C++ 开发经验、熟悉 Lua，并已完成前三课。

本课目标：把“能跑的 Skynet MMO 样例”放进真实研发与上线体系，建立可观测、可复现、可定位、可压测、可回滚的工程闭环。

基线：Skynet v1.8.0、Skynet Bundled Modified Lua 5.4.7、Linux/WSL2、Proto + 2-byte big-endian framing。

SKYNET MMO LEARNING

目录

0. 先说结论：生产能力不是“线上能不能下断点”	5
1. 四类环境，四种操作权限	6
1.1 个人开发环境	6
1.2 共享开发环境	6
1.3 Staging / 压测环境	6
1.4 Production	6
2. 先建立证据模型，而不是先选工具	7
3. 结构化日志：从文本说明升级为可关联事件	8
3.1 事件模型	8
3.2 日志级别和采样	8
3.3 必须记录 yield 前后的版本	8
3.4 敏感信息	8
4. Trace：沿 Skynet 消息链保留因果关系	9
4.1 三种 ID 不要混用	9
4.2 当前业务链的传播点	9
4.3 官方消息 Trace 能做什么	9
5. Metric：从“CPU 高”拆成可行动结论	10
5.1 Service 级最小指标集	10
5.2 低基数原则	10
5.3 延迟必须是分布	10
6. Skynet 排队和调度：延迟应如何分解	11
6.1 Little's Law 用于粗验	11
7. Debug Console：命令、证据与风险	12
7.1 推荐的只读诊断顺序	12
7.2 会改变状态的命令	12
7.3 task 如何定位同步等待链	12
8. Endless Detection：它能发现什么，不能发现什么	14
9. LuaPanda、GDB、Core Dump 的分工	15
9.1 LuaPanda	15
9.2 GDB Live Debug	15

9.3 Core Dump	15
9.4 Lua Stack 与 C Stack 的关联	16
10. CPU Profiling: 先回答 “在哪烧 CPU”	17
10.1 stat 只能定位 Service, 不够定位函数	17
10.2 Linux perf	17
10.3 优化顺序	17
11. 内存、GC 与 jemalloc: 四个口径必须分开	18
11.1 四层内存	18
11.2 判断模式	18
11.3 gc 不是无害的清理按钮	18
12. Socket 与协议诊断	19
13. Record/Replay: 把偶现时序问题变成确定性测试	20
13.1 只录包还不够	20
13.2 确定性竞态实验	20
14. 测试体系: 正确性、故障和性能分开验收	21
14.1 建议分层	21
14.2 故障注入原则	21
15. Benchmark: 当前 AOI 脚本测到了什么	22
16. 真实 MMO 负载模型	23
16.1 压测报告的最低格式	23
17. Hot Actor、分片与背压	24
17.1 何时拆 Scene	24
17.2 背压不是无限加 Queue	24
18. Storage 可观测性与一致性	25
19. 发布、灰度和回滚	26
19.1 发布物必须可重建	26
19.2 灰度不是只启动一台新版本	26
19.3 Graceful Shutdown	26
20. 安全边界	27
21. 四个现场故障剧本	28
21.1 P99 延迟升高, 但 CPU 不高	28

21.2 单核打满、Scene Mailbox 增长	28
21.3 RSS 持续增长	28
21.4 玩家重连后被旧 Agent 踢下线	28
22. 当前仓库能力矩阵	29
23. 实验一：建立一份 Service 诊断快照	30
24. 实验二：定位一次人为慢调用	31
25. 实验三：做一份合格 AOI Before/After 报告	32
26. 实验四：Core Dump 演练	33
27. 生产变更审查模板	34
28. 上线门禁	35
正确性	35
可观测性	35
性能	35
发布与恢复	35
29. 自测题	36
29.1 语义题	36
29.2 设计题	36
29.3 实操验收	36
30. 源码与命令索引	37
31. 四课完成后，你应具备的完整运行时地图	38

0. 先说结论：生产能力不是“线上能不能下断点”

个人开发环境当然应当能下断点。本仓库已经用 Windows VS Code + WSL2 + LuaPanda 完成 Scene 和 PlayerAgent 的 Lua 断点闭环，也能用 GDB 调试 Native Runtime。

但生产系统的核心要求不是把 IDE 搬进线上，而是：

1. 故障发生前已有低开销证据；
2. 故障发生后能沿一条请求还原消息链和状态变化；
3. 不能复现时，仍能用指标、Trace、Core Dump 和 Profile 缩小范围；
4. 修复必须变成确定性测试或回放用例；
5. 发布异常时可以停止扩散并回滚。

这和成熟 C++ MMO Server 的工作方式没有本质差异。Skynet 的特殊点在于：调度单位、排队位置和状态竞争不再只落在 OS Thread、锁和对象上，还落在 Service Mailbox、消息 coroutine、`skynet.call` 的 session 以及 yield/resume 边界上。

本课完成后，你应能回答：

- 延迟来自 Gate 排队、目标 Service Mailbox、handler 执行、下游 RPC，还是 Socket 写缓冲？
- 某个 Service CPU 高，是消息量高、单消息慢、死循环，还是 GC？
- RSS 上升，是 Lua Heap、Skynet C Allocation、Socket Buffer，还是 jemalloc 保留页？
- 某次重连为何被旧 offline coroutine 删除？如何留下可回放证据？
- 一个 AOI 优化为何在微基准里变快，却让真实帧尾延迟恶化？
- 哪些诊断动作可以在线上执行，哪些只能在个人或隔离 Staging 使用？

1. 四类环境，四种操作权限

1.1 个人开发环境

目标是快速理解控制流和状态。

允许使用：

- LuaPanda 断点、条件断点、Variables、Watch；
- GDB 启动调试和断点；
- Debug Console 全部只读命令；
- 确定性故障注入；
- Debug Build、额外断言、详细 Trace；
- 单元测试、集成测试、回放和微基准。

允许暂停单个 Service，因为该环境没有其他研发共用流量。即使如此，也要记住：LuaPanda 暂停的是正在执行该 Lua State 的路径，其他 Worker Thread 与其他 Service 仍可能前进。

1.2 共享开发环境

目标是多人联调，不破坏其他人的会话。

默认允许：结构化日志、有限 Trace、只读 `stat/task/info/netstat/mem`、指标和按玩家隔离的回放。断点仅用于专属实例，不应附加到共享进程。`gc`、`inject`、`kill`、`exit` 等改变运行状态的命令必须经过值班/环境所有者确认。

1.3 Staging / 压测环境

目标是复现生产拓扑、发布物和流量形态。

允许在受控窗口使用 `perf`、GDB attach、Core Dump、短时高采样 Trace、jemalloc Heap Profiling 和故障注入。测试结束必须恢复采样率、关闭 Trace，并保存构建 ID、配置版本和采样条件。

1.4 Production

目标首先是控制影响面，其次才是定位。

默认工具是低基数指标、结构化日志、采样 Trace、健康检查、受控状态快照、Core Dump 和已评估开销的 Profiler。LuaPanda、任意 `inject`、无限制消息 Trace、公开 Debug Console 都不属于生产方案。

生产诊断按以下顺序升级：

- Dashboard / Alert
 - > 定位节点、进程、Service、玩家与时间窗
 - > 查看结构化日志和采样 Trace
 - > 只读状态快照 / Mailbox / task
 - > 短时提高采样或受控 Profile
 - > 隔离实例、摘流量、GDB/Core Dump
 - > 回放 + Regression Test

2. 先建立证据模型，而不是先选工具

完整证据分为六类：

证据	回答的问题	典型工具
Log	发生了什么业务事件	Structured Log
Metric	规模、趋势和异常时间窗	Counter/Gauge/Histogram
Trace	一次请求跨过了哪些 Service	Trace/Context ID
Profile	CPU 或 Allocation 花在哪里	<code>perf</code> 、Flame Graph、Heap Profile
Dump	崩溃或冻结瞬间的完整进程状态	Core Dump + GDB
Recording	输入、时序和外部结果能否重演	Packet/Command Record-Replay

不能让一种证据承担所有职责。日志不适合聚合 P99，Metric 不保存每个玩家的业务细节，Trace 不等于可重放，Profile 也不能证明状态一致性。

建议所有证据至少关联这些身份：

```

build_id / config_version / node / process_id
service_handle / service_name
trace_id / request_id / operation_id
account_id / player_id (按隐私规范脱敏)
connection_id / fd_generation
scene_id / scene_epoch

```

`fd` 不能独立作为连接身份。文件描述符会复用；必须与连接 `generation/session` 组合。Service Handle 也可能随进程重启变化，应同时记录 `node`、`build ID` 和逻辑服务名。

3. 结构化日志：从文本说明升级为可关联事件

3.1 事件模型

不要写只有人能读的：

```
skynet.error("save failed")
```

生产日志至少要表达事件、主体、版本、耗时和结果。下面是目标形态，不代表仓库已经实现 JSON Logger：

```
{
  "level":"ERROR",
  "event":"player_save_failed",
  "trace_id":"01J...",
  "operation_id":"logout:10001:42",
  "player_id":10001,
  "agent":"00000018",
  "state_version":42,
  "storage_shard":3,
  "latency_ms":126,
  "error":"timeout",
  "build_id":"git:abc1234"
}
```

字段名和类型必须稳定，避免同一字段有时是字符串、有时是数值。事件名称应代表业务事实，例如 `login_rejected`、`scene_enter_committed`、`player_save_failed`，而不是源码函数名。

3.2 日志级别和采样

- **ERROR**：需要处理的失败；不能把客户端参数错误全部打成 ERROR。
- **WARN**：系统可以继续，但出现退化、重试、慢调用或容量风险。
- **INFO**：低频生命周期和关键业务状态迁移。
- **DEBUG/TRACE**：个人或短时诊断，默认不进入生产全量流。

高频移动每包打 INFO，会同时制造 I/O、格式化、锁竞争、磁盘容量和检索成本。更合理的是：移动结果计数做 Metric；异常移动记录采样日志；指定玩家用动态 Trace；必要时录制协议流。

3.3 必须记录 yield 前后的版本

以第三课发现的 offline/reconnect 窗口为例：

```
offline_begin player=10001 offline_version=7 client_generation=12
scene_leave_begin operation_id=offline:10001:7
scene_leave_end ... current_offline_version=8
offline_abort reason=version_changed
```

关键不是多打印两行，而是让日志能够证明：coroutine 在 `call Scene.leave` 处 yield，恢复时看到的 version 是否仍与开始时一致。

3.4 敏感信息

Token、密码、完整 Session Key、支付凭据和数据库连接密码不得进入日志。Player ID 是否属于个人信息取决于业务规范；对外部平台和长期归档应脱敏。错误对象必须经过字段白名单，不能把整份请求随手序列化。

4. Trace：沿 Skynet 消息链保留因果关系

4.1 三种 ID 不要混用

- `trace_id`：一条端到端因果链，例如一次 Login 或 Attack。
- `request_id`：一次协议请求，可用于客户端重试关联。
- `operation_id`：有业务幂等含义的操作，例如发奖、扣费、保存版本。

一次请求可能产生多个异步消息，仍共享 `trace_id`；重试会产生新的 transport request，但幂等操作应保留同一 `operation_id`。

4.2 当前业务链的传播点

```
Client packet
-> Gate/Watchdog: 创建或接受 trace_id, 绑定 connection_generation
-> PlayerAgent: 加入 player_id、agent handle、state_version
-> Scene: 加入 scene_id、scene_epoch、entity_version
-> Storage: 加入 shard、snapshot_version、db operation_id
-> Response/Push: 保留 trace_id 与 result
```

Skynet 的消息封包不会自动理解业务 Context。真实项目需要在协议 Envelope、内部 RPC 参数或 coroutine-local context 中显式传播，并为 `skynet.fork`、timer callback 和 `send` 规定继承规则。

4.3 官方消息 Trace 能做什么

本仓库的 Skynet v1.8.0 在 `lua-lib/skynet.lua` 中实现 `skynet.trace`、`tracetag`、`tracecall` 与协议级 `traceproto`；官方 `service/debug_console.lua` 的 `trace` 命令通过 debug protocol 调用目标 Service 的 `TRACELOG`。

```
trace :00000012 lua on
trace :00000012 lua off
```

它适合在短时间窗确认消息的 request/call/response/resume 关系，但不是完整分布式追踪平台。开启整个高频 Service 的全量 Trace 会放大日志量和运行开销。生产应限制 Service、玩家、协议、采样率和持续时间，并确保自动关闭。

5. Metric：从“CPU 高”拆成可行动结论

5.1 Service 级最小指标集

每个逻辑 Service 类型至少应有：

- Mailbox 当前长度、最大长度、增长速率；
- 消息吞吐量，按 protocol/command 受控聚合；
- Handler 执行耗时分布；
- `call` 等待耗时分布及目标类型；
- 活跃 coroutine 数、超时/取消数；
- Lua Heap、GC 次数、GC Pause/Step 时间；
- Error/Timeout/Reject/Retry 计数；
- Service 启停和异常退出计数。

Skynet 官方 `skynet/debug.lua` 的 `STAT` 已返回：

```
task / mqlen / cpu / message
```

其中 `mqlen` 来自当前 Service 私有 Mailbox；`cpu` 是 Service 累积 CPU 时间；`message` 是累计处理消息数。它们适合诊断快照。生产监控仍需周期采集、差分并写入时序系统，才能得到速率和趋势。

5.2 低基数原则

`service_type=scene` 是可控维度，`player_id=10001` 通常不应成为 Metric Label。把百万玩家 ID 放进时序标签会造成 Cardinality Explosion。玩家级定位留给 Log/Trace，Metric 只保留有限的 Region、Node、Service Type、Command、Result。

5.3 延迟必须是分布

平均延迟会掩盖 MMO 最敏感的长尾。至少报告 P50/P95/P99/Max 和样本量，并明确时间窗。Histogram Bucket 应围绕业务 SLO 设计，例如：1、2、5、10、20、50、100、200、500 ms。

6. Skynet 排队和调度：延迟应如何分解

一条 `move` 的端到端延迟可以写成：

```
T_total = T_socket_read
          + T_gate_queue + T_gate_exec
          + T_agent_queue + T_agent_exec_before_call
          + T_scene_queue + T_scene_exec
          + T_response_queue
          + T_socket_write
```

若 `PlayerAgent` 执行 `skynet.call(Scene, "lua", "move", ...)`，它的 `coroutine` 会等待 `Response`；同一 `PlayerAgent` 仍可处理其他消息。于是延迟诊断必须同时观察：

1. 调用方 `coroutine` 在等谁；
2. 目标 `Service Mailbox` 是否积压；
3. 目标 `handler` 实际 `CPU` 时间；
4. 目标是否继续 `call` 第三个 `Service`；
5. 恢复时调用方状态是否已被别的消息改变。

这比 `C++ Thread-per-connection` 模型多了一层显式 `Mailbox` 与 `coroutine session`，但思路类似于拆解线程池排队、锁等待、下游 `RPC` 和执行时间。

6.1 Little's Law 用于粗验

稳定系统近似满足：

$$L = \lambda \times W$$

如果 `Scene` 每秒接收 10,000 条消息，平均在系统中停留 20 ms，则平均在途量约 200。若 `Mailbox` 持续增长而吞吐不变，系统已经不稳定；提高客户端超时只会扩大在途请求和内存占用。

7. Debug Console：命令、证据与风险

当前开发配置 `config/game.lua` 启动 `debug_console_port = 8000`。连接方式：

```
./scripts/linux/debug_console.sh
```

7.1 推荐的只读诊断顺序

```
list
stat
task :00000012
info :00000012
ping :00000012
netstat
mem
cmem
jmem
```

- **list**：Launcher 中的 Service 实例和地址。
- **stat**：各 Lua Service 的 `task/mqlen/cpu/message` 快照。
- **task**：目标 Service 中挂起 coroutine 的 traceback；适合找同步 RPC 等待链。
- **info**：调用 Service 自己注册的 `skynet.info_func`；没有注册时返回空。
- **ping**：测 Console 到目标 Service 的调度和应答时间，不等于客户端 RTT。
- **netstat**：Socket 状态、读写量、写缓冲和最近读写时间。
- **mem**：各 Lua Service 的 Lua Heap 估算。
- **cmem**：Skynet C Allocation 的 Service 归属与总量。
- **jmem**：jemalloc 统计；它与 Lua Heap、进程 RSS 不是同一个口径。

7.2 会改变状态的命令

`gc`、`trace`、`logon/logoff`、`profactive/dumpheap` 会改变开销或输出；`inject`、`call`、`killtask`、`exit`、`kill`、`start` 会执行代码或改变生命周期。它们不是“只是 Console 命令”。生产权限必须分级、审计，并限制到本机或管理网络。

尤其禁止把当前 8000 端口直接暴露公网。正式配置应默认关闭；需要时绑定 loopback/Unix domain socket 或管理平面，并由防火墙、身份认证和操作审计保护。当前仓库只有教学用 TCP Console，没有实现完整生产访问控制。

7.3 task 如何定位同步等待链

假设 PlayerAgent 卡住：

```
task :AGENT
player_agent.lua -> skynet.call(Scene)

task :SCENE
scene.lua -> skynet.call(Storage)

stat
Storage mqlen 持续增长
```

这构成“Agent 等 Scene，Scene 等 Storage，Storage 排队”的证据。不要看到 Agent Mailbox 高就直接扩 Agent；根因可能是更深的同步链。

8. Endless Detection: 它能发现什么，不能发现什么

`skynet_start.c` 创建独立 Monitor Thread。它每 5 秒检查各 Worker 的 monitor version；若同一 Worker 正在处理的 source/destination 长时间没有变化，`skynet_monitor.c` 会输出：

```
A message from [source] to [destination] maybe in an endless loop
```

并把目标 Service 的 `endless` 标记设为 true。Lua 层 `skynet.endless()` 读取并清除此标记。

这更接近 Worker 长时间困在一次不 yield 的消息处理，而不是一般意义的 coroutine deadlock。以下情况不一定触发同一种告警：

- coroutine 正在等待永不返回的 `skynet.call`；它已经 yield；
- Mailbox 不断增长，但单个 handler 都能完成；
- 业务逻辑不断 fork 短 coroutine；
- Socket 或数据库外部依赖变慢。

因此 Endless Log 必须与 `task`、Mailbox、CPU Profile 和下游延迟结合。

9. LuaPanda、GDB、Core Dump 的分工

9.1 LuaPanda

适合个人环境逐行观察指定 Lua Service 的局部控制流。本仓库只在 `config/debug_luapanda.lua` 下、只对显式选择的 Service 注入。正常、测试、生产路径均不加载。

它不适合生产的原因不是“不专业”，而是断点会改变调度时序；Debug Hook 和 LuaSocket 有额外开销；暂停期间其他 Service 仍然运行；多个同名 Service 还会竞争调试端口。

9.2 GDB Live Debug

GDB 适合：Native Crash、C Module、Socket Thread、Worker Thread、死锁/阻塞和 Runtime 数据结构。个人环境可以用 VS Code 的 C/C++ Configuration 启动 Skynet；Staging 可在摘流量后 attach。

常用命令：

```
set pagination off
info threads
thread apply all bt full
break skynet_context_message_dispatch
break skynet_error
continue
```

生产 live attach 会暂停全部线程片刻，并可能在高负载进程中扩大抖动。必须先确认实例冗余、摘流量策略、ptrace 权限和操作窗口。

9.3 Core Dump

Core Dump 保存崩溃瞬间的所有线程、寄存器、Native Stack 和映射。它不会自动给出 Lua 业务语义，但能确认 Signal、Faulting Thread、C Stack、共享库和内存破坏迹象。

实验环境示意：

```
ulimit -c unlimited
./third_party/skynet/skynet config/game.lua
```

若系统使用 systemd-coredump：

```
coredumpctl list skynet
coredumpctl info skynet
coredumpctl debug skynet
```

若直接生成 core 文件：

```
gdb ./third_party/skynet/skynet /path/to/core
```

进入 GDB 后：

```
info threads
thread apply all bt full
info sharedlibrary
info proc mappings
```

必须保存与 Core 完全匹配的 ELF、未剥离符号、C Module、Lua C Module、Build ID 和配置。用另一个 commit 的二进制分析 Core，行号和结构布局都可能错误。

9.4 Lua Stack 与 C Stack 的关联

GDB 能直接看到 `lua_pcallk`、Skynet dispatch 等 Native Frame，却未必自然展开每个 Lua coroutine 的业务栈。实际工程应：

1. 在正常运行期用 `task` 保存挂起 coroutine traceback；
2. 在错误边界记录 Lua traceback、Service、source/session 和 `trace_id`；
3. 为指定 Lua 版本准备并版本化 GDB Lua State 辅助脚本；
4. 用 Core 的 Worker/Service 信息与日志时间线做关联。

辅助脚本必须针对本仓库 Bundled Modified Lua 5.4.7 验证，不能拿 LuaJIT 或 Lua 5.1 的脚本直接套用。

10. CPU Profiling: 先回答“在哪烧 CPU”

10.1 stat 只能定位 Service, 不够定位函数

stat 的 cpu 是累计值。两次采样的差值除以时间窗, 可粗看哪个 Service 消耗 CPU; 再用消息数差值得到每消息平均 CPU。但平均值仍可能掩盖个别慢路径。

10.2 Linux perf

在 Staging 使用与生产一致的 Release Build 和符号包:

```
perf stat -p <pid> -- sleep 30
perf record -F 99 -g -p <pid> -- sleep 30
perf report
```

关注:

- 采样窗口是否覆盖问题;
- Frame Pointer / DWARF Unwind 是否可用;
- Native Symbol 是否匹配;
- 99 Hz 是否足够且开销可接受;
- 是否把 Logger、Allocator、协议编解码和业务函数分开。

Native Flame Graph 很适合看 Skynet C Runtime、Lua VM、Sproto C Module、jemalloc 和系统调用, 但默认不会自动显示漂亮的 Lua 函数名。要做 Lua 函数级 CPU Profile, 需要接入与 Lua 5.4.7/Skynet coroutine 模型兼容的已维护 Profiler, 并先验证采样准确性、coroutine 归属和开销。

本仓库当前没有提供已验收的 Lua 函数级 Profiler。compat10/profile.lua 只是兼容入口, 目标 skynet.profile 模块并不在本仓库发布物中, 不能据此宣称已经具备该能力。

10.3 优化顺序

```
先用指标定位 Node/Service/Command
-> 用 Trace 确认调用链和输入类型
-> 用 Profile 找函数与分配热点
-> 做最小改动
-> 同负载、同机器、同构建参数复测
-> 检查 P99、CPU、内存和正确性回归
```

不要看到 Lua 就先改 C; 也不要凭经验把 Scene 拆成更多 Service。Profile 可能证明真正热点是序列化、日志、分配、错误重试或不合理的同步 RPC。

11. 内存、GC 与 jemalloc：四个口径必须分开

11.1 四层内存

1. Lua Heap：Lua Object、String、Table、Closure 等；Debug Console **mem** 可看 Service 维度估算。
2. Skynet C Allocation：消息、Context 和由 malloc hook 追踪的内存；**cmem** 给出 Service 归属与总量。
3. jemalloc Arena/Retention：已向 OS 申请但未必仍被业务对象占用；**jmem** 查看 allocator 统计。
4. Process RSS：OS 看到的 Resident Pages，还包括 Stack、Code、Shared Library、Socket Buffer 映射等。

因此“Lua GC 后 RSS 没降”不等于泄漏。对象可能已释放给 jemalloc，但页面仍保留供后续复用。正确做法是同时画出 Lua Heap、C Allocation、jemalloc active/resident/retained 与 RSS 的时间序列。

11.2 判断模式

现象	初步方向
Lua Heap 持续升，强制 GC 后仍不降	Lua Object 被引用
Lua Heap 锯齿，RSS 高位稳定	正常 GC + allocator 保留可能性
cmem 上升，Lua Heap 不升	C Module/消息/Buffer 路径
Mailbox 与内存一起升	消费能力不足或下游阻塞
Socket wbuffer 升	慢客户端/网络背压

11.3 gc 不是无害的清理按钮

官方 debug GC 会对目标 Service 执行完整 `collectgarbage("collect")`，随后主动 yield，并记录耗时。Debug Console 的 **gc** 会请求每个 Lua Service GC。大 Heap 上全局强制 GC 可能制造延迟尖峰，只应在受控诊断窗口使用。

profactive/dumpheap 依赖 jemalloc Heap Profiling 的构建和运行配置。命令存在不代表当前二进制已经启用全部采样能力；使用前必须在 Staging 验证产物、输出路径、磁盘容量和开销。

12. Socket 与协议诊断

`netstat` 返回 Socket 的 read/write、`wbuffer`、最近读写时间和地址等信息。诊断慢客户端时优先看：

- 写缓冲是否持续增长；
- 最近写时间是否远早于当前时间；
- Gate 是否继续向该 fd 推送；
- fd 对应的 connection generation 是否仍是当前连接；
- 断线后旧消息是否能误发给复用 fd。

当前协议使用 2-byte big-endian 长度头。包录制至少保存：方向、单调时钟、连接 generation、原始 frame、解码后的 protocol/version、trace_id 和构建版本。敏感字段应在录制前脱敏或加密，并设置保留期限。

协议错误要分层：

```
TCP stream/framing error
-> Sproto decode/schema/version error
-> authentication/session error
-> command validation error
-> state/version conflict
-> internal execution error
```

把所有失败都返回 `INTERNAL_ERROR` 会丢掉容量与安全信号；把内部 traceback 原样发给客户端又会泄露实现细节。

13. Record/Replay：把偶现时序问题变成确定性测试

13.1 只录包还不够

要重放第三课的 offline/reconnect 竞态，需要记录或控制：

- 入站命令与到达顺序；
- connection/session generation；
- `skynet.now/timer` 的逻辑时间；
- 随机种子；
- Scene/Storage RPC 的完成点和结果；
- 初始 Player Snapshot 与版本；
- Build ID、Config Version、协议版本。

外部数据库若不隔离，Replay 结果会被实时状态污染。应把外部结果录制为 Fixture，或在隔离数据库中构造相同初始版本。

13.2 确定性竞态实验

1. Agent offline_version = 7，启动 offline coroutine
2. Scene.leave 在测试屏障处挂起
3. 新连接执行 bind_client，使 version = 8
4. 释放 Scene.leave
5. 断言旧 offline coroutine 在恢复后重新校验并退出
6. 断言 Agent 未被 PlayerMgr.remove，Scene/Storage 状态符合协议

这类 Test Hook 只进入测试配置，不能通过真实 `sleep` 猜测竞态窗口。每个线上时序 Bug 的最终产物应包括：最小录制、确定性 Regression Test、修复和监控项。

14. 测试体系：正确性、故障和性能分开验收

当前 `./scripts/linux/test.sh` 已覆盖 Pure Lua Unit Test、LuaPanda Tooling Regression 和真实 Gate/Sproto Integration Smoke。它证明基础路径可运行，但不能替代生产测试矩阵。

14.1 建议分层

层级	目标	例子
Unit	纯算法/状态机边界	AOI、伤害、移动校验、Version Compare
Service Contract	单 Service 消息契约	Agent 重复请求、Scene stale epoch
Integration	真实 Runtime 和协议链	Login-Move-Attack-Logout
Failure	下游超时、退出、重连、背压	Storage timeout、Scene restart
Replay	固化线上/偶现输入	offline/reconnect window
Load/Soak	容量、长尾和泄漏	高密 Scene、慢客户端、8h Soak

14.2 故障注入原则

注入点应是明确的协议边界：RPC 调用前、下游提交后响应前、timer 触发前、Socket 写入前。每个注入带 operation ID，默认关闭，只在测试配置启用。禁止用宽泛 `pcall` 吞掉失败来模拟“容错”。

15. Benchmark：当前 AOI 脚本测到了什么

现有命令：

```
./scripts/linux/benchmark_aoi.sh 10000 100000
```

`tests/benchmark/aoi_bench.lua` 固定随机种子 `20260911`，构造 10,000 个 Entity，对 3x3 Grid Candidate Query 执行 100,000 次，输出 elapsed、QPS、Average Candidate 和 Cell Count。

它适合比较 AOI 数据结构改动，不是服务器容量数字，原因包括：

- 使用 `os.clock` 测 CPU 时间，不包含真实 Socket、Sproto 和调度链；
- Entity 近似均匀随机，不代表主城热点；
- 只 Query，不覆盖 Move、Enter/Leave、广播和 Allocation 生命周期；
- 没有并发 Scene、GC、日志和 Storage 干扰；
- 只输出平均 Candidate，没有延迟分位数。

AOI 变更必须保留同机器、同构建、同参数的 before/after：

```
commit / CPU / kernel / build flags / Lua version  
entity distribution / query count / warmup / repetitions  
P50/P95/P99 or per-batch distribution  
CPU time / wall time / allocations / correctness result
```

按仓库规则，AOI Algorithm Change 除完整测试外必须运行 `benchmark_aoi.sh` 并报告前后数字。

16. 真实 MMO 负载模型

不能只说“10 万机器人”。负载模型至少包含：

- 登录/登出速率，而不仅是在线峰值；
- 玩家在线时长和重连分布；
- Move、Attack、Chat、Inventory 的命令占比；
- 主城/副本/野外的空间密度分布；
- 广播 Fan-out 和可见集大小；
- 慢客户端、丢包、半开连接；
- Storage 延迟和错误分布；
- 定时任务同一秒集中触发；
- 灰度期间新旧协议共存。

一个粗略容量例子：

```
20,000 CCU
每玩家平均 5 move/s = 100,000 move/s
10% 玩家处于战斗，每人 2 combat command/s = 4,000/s
平均可见 40 人，每次位置广播若不聚合，潜在 fan-out = 4,000,000 push/s
```

这说明入口请求量可能不是瓶颈，广播放大才是。必须测编码 CPU、消息分配、Socket wbuffer、带宽和慢客户端淘汰策略。

16.1 压测报告的最低格式

```
目标 SLO 与失败阈值
拓扑、实例数、Worker Thread、CPU/Memory/Network
Build ID、Config Version、数据集、机器人版本
Ramp-up / Steady / Cool-down 时长
CCU、输入 RPS、Push/s、Traffic
P50/P95/P99/Max、Error/Timeout/Reject
每类 Service Mailbox、CPU、Lua/C/Jemalloc/RSS
瓶颈证据、第一饱和点、恢复行为
```

容量应取满足 SLO 且留有故障冗余的负载，不取压到崩溃前最后一个漂亮 QPS。

17. Hot Actor、分片与背压

17.1 何时拆 Scene

拆分依据是持续证据：Scene Mailbox、每消息 CPU、AOI Fan-out、Tick Overrun 和跨核利用率，而不是“Actor 应该越多越好”。

可选策略：

- 按副本/地图天然分 Scene；
- 热门大地图按 Region/Cell 分区；
- 把可异步的低频旁路从数据面移出；
- 合并同 Tick 内 Position Update 和 Broadcast；
- 对非关键 Push 降采样或丢弃旧版本。

分区会引入跨区移动 Saga、Ghost Entity、Order、Epoch 和更多消息，必须用收益覆盖复杂度。

17.2 背压不是无限加 Queue

Mailbox 增长时需要明确策略：

- Login 入口限流和快速拒绝；
- 每连接命令速率与 Burst 限制；
- 同玩家可合并的 Position Update 只保留最新版本；
- 给 RPC 设置业务 Deadline，并在下游拒绝过期工作；
- Storage Pool 达到高水位时停止接收非关键 Flush；
- 慢客户端写缓冲超限时断开或降级 Push。

关键交易命令不能静默丢弃；必须通过 Idempotent Operation、Journal/Ledger 和明确错误恢复。

18. Storage 可观测性与一致性

内存或 MySQL Driver 至少要暴露：

- Pool Queue、In-flight、连接数；
- Query/Transaction Latency 分布；
- Timeout、Disconnect、Retry、Conflict；
- Snapshot Version、Dirty Age、Save Batch Size；
- Slow Query 指纹，而非完整敏感 SQL；
- 最后成功保存时间和未保存玩家数。

重试不能只看网络错误。若数据库已经 Commit、Response 丢失，调用方重试会重复执行。金币、道具、支付等路径需要稳定 `operation_id` 与唯一约束/Journal，普通 Snapshot 保存需要 Compare-and-Swap Version 或等价协议。

不要在 PlayerAgent 中同步串联深链：

Agent -> Scene -> Inventory -> Guild -> Storage

任何下游抖动都会沿 `call` 放大。设计审查必须重新问：结果是否需要立即返回？`send` 是否足够？状态 Owner 能否本地提交后异步发布？失败如何补偿？

19. 发布、灰度和回滚

19.1 发布物必须可重建

每次发布保存：

- Git Commit 与 Dirty State；
- Skynet/Lua 固定版本；
- 编译器、Build Flags、Native Dependency；
- ELF Build ID 与独立 Debug Symbols；
- Config/Schema/Protocol Version；
- 数据迁移版本和回滚条件；
- SHA-256 Manifest。

本仓库固定 Skynet v1.8.0，不允许在业务改动中静默升级 Runtime。

19.2 灰度不是只启动一台新版本

灰度计划要说明：流量选择、观察窗口、关键 SLO、错误预算、自动停止条件、状态兼容和回滚后旧进程能否读取新数据。涉及 Player State Schema 时，代码回滚并不自动等于数据回滚。

19.3 Graceful Shutdown

理想顺序：

- 停止新连接/登录
 - > 从服务发现或负载均衡摘除
 - > 等待或迁移活跃会话
 - > 停止产生新的非关键任务
 - > Flush Dirty Snapshot / Journal
 - > 验证未完成 operation
 - > 按依赖顺序退出 Service
 - > 设置 Deadline，超时则保留恢复证据

当前样例尚未实现完整 Drain、批量保存、重启恢复和 State Migration，这些仍属于生产 Gap。

20. 安全边界

以下接口应视为远程代码执行或高权限运维面：

- Debug Console `inject/call/start/kill/exit`;
- GDB/ptrace;
- Core Dump (包含内存中的 Token、玩家数据和数据库凭据) ；
- Packet Recording;
- Heap Dump;
- 动态配置和 Hotfix。

必须有网络隔离、身份认证、最小权限、操作审计、加密存储和保留/销毁策略。Core 与录包不能随意上传到普通聊天或公共 Issue。

当前 `config/game.lua` 中包含教学用数据库账号密码并监听 `gate_host = "0.0.0.0"`；它不是生产 Secrets Management 示例。生产必须使用 Secret Store/受控环境注入，并区分开发默认值与正式启动校验。

21. 四个现场故障剧本

21.1 P99 延迟升高，但 CPU 不高

排查顺序：

1. 确认受影响命令、节点、Scene 和时间窗；
2. 对比各 Service Mailbox 与 `task`；
3. 拆 Agent/Scene/Storage RPC Latency；
4. 查看 Storage Pool、DB Slow Query 和 Socket wbuffer；
5. 看是否出现同步链、重试风暴或 Timer Burst；
6. 采样 Trace 还原一条慢请求；
7. 在 Staging 注入同样下游延迟验证。

CPU 不高并不代表系统空闲，可能大多数 coroutine 都在等待下游。

21.2 单核打满、Scene Mailbox 增长

1. `stat` 找累计 CPU 和消息增量异常的 Scene；
2. 看命令分布与 AOI Density；
3. `perf` 找 Native/Lua VM/Encoding/Allocation 热点；
4. 检查是否出现 Endless Log；
5. 用真实热点分布复现；
6. 优化后同时比较 P99、CPU、Mailbox、Allocation 和 Correctness。

21.3 RSS 持续增长

1. 同图对齐 Lua `mem`、`cmem`、`jmem`、RSS、Mailbox、Socket wbuffer；
2. 区分对象保留、C Allocation、Allocator Retention 与积压；
3. 不要先在线上全局 `gc`；
4. 在 Staging 对相同负载做 Heap Profile/Soak；
5. 若是 Crash 前增长，保留 Core 与匹配符号。

21.4 玩家重连后被旧 Agent 踢下线

1. 用 `player/connection/offline version` 关联日志；
2. Trace offline coroutine 的每个 `call` 和恢复点；
3. 确认新 `bind_client` 是否在 yield 窗口进入；
4. 录制 Scene.leave/Storage.save/PlayerMgr.remove 的完成顺序；
5. 构造测试屏障稳定复现；
6. 每次 await 恢复重新校验 generation/version；
7. 将案例固化为 Regression Test。

这正是第三课记录的当前已知 Gap；本课只建立诊断与验收方法，不声称已经修复。

22. 当前仓库能力矩阵

能力	当前状态	证据/入口
Native Build + 固定 Runtime	已有	<code>scripts/linux/build.sh</code>
Unit + Integration Smoke	已有	<code>scripts/linux/test.sh</code>
AOI Microbenchmark	已有	<code>scripts/linux/benchmark_aoi.sh</code>
Lua 图形断点	已有, Dev-only	<code>config/debug_luapanda.lua</code> 、 <code>docs/16_DEBUGGING.md</code>
C Runtime 调试	已有开发入口	VS Code/GDB 配置
Debug Console	已有教学入口	<code>scripts/linux/debug_console.sh</code>
Mailbox/Task/Memory 快照	上游能力可用	<code>stat/task/mem/cmем/jmem</code>
官方消息 Trace	可手动使用	<code>trace <address> lua on/off</code>
Structured Log Pipeline	未实现	生产 Gap
Metrics + Alerting	未实现	生产 Gap
跨 Service Context Propagation	未实现	生产 Gap
Packet Record/Replay	未实现	生产 Gap
Lua 函数级 Profiler	未验收/未提供	需选型并验证 Lua 5.4.7 + coroutine
Core Dump 自动收集和符号仓库	未实现	生产 Gap
Load/Soak Harness	未实现	只有 AOI Microbenchmark
Graceful Drain/Recovery	未实现	生产 Gap
灰度/自动回滚	未实现	生产 Gap

“命令能运行”与“生产能力已完成”必须严格区分。例如 `jmem` 命令存在，只说明 Runtime 暴露接口；正式使用仍要验证构建选项、采样开销、存储和分析流程。

23. 实验一：建立一份 Service 诊断快照

启动服务器和客户端，完成登录、移动、攻击后连接 Debug Console：

```
list
stat
task :<player_agent_handle>
task :<scene_handle>
netstat
mem
cmem
jmem
```

输出不只是截图，应整理成：

```
采样时间与 Build ID
Client fd + generation -> Agent Handle
Agent -> Scene -> Storage 的同步等待关系
各 Service mqlen/cpu/message
Socket wbuffer
Lua/C/jemalloc 三种内存口径
观察结论与尚不能证明的部分
```

验收：能够解释每个数字在哪层产生、是累计量还是瞬时量、为什么不能由一次采样推导趋势。

24. 实验二：定位一次人为慢调用

只在测试配置中给 Storage Worker 增加可控 Barrier：请求到达后挂起，由测试进程释放。不要用固定 `sleep`。

观察：

1. Agent/Scene coroutine 在哪里 yield；
2. `task` 是否能看出等待链；
3. Storage Mailbox 是否增长；
4. CPU 是否仍然较低；
5. 客户端 timeout 后，旧请求恢复是否还会写状态；
6. Deadline、Version、Operation ID 应在哪层校验。

最终产物是 Integration Regression Test，而不是人工操作说明。

25. 实验三：做一份合格 AOI Before/After 报告

即使不修改算法，也执行多次：

```
./scripts/linux/benchmark_aoi.sh 10000 100000
```

记录机器、CPU Governor、WSL/Native Linux、Commit、运行次数、Warmup、Median 和离散程度。
再构造均匀、主城热点、沿边界移动三种数据分布。

验收不是“QPS 更高”，而是：

- Correctness Test 全通过；
- 明确改动优化了 Query、Update、Allocation 中哪一项；
- 解释 Candidate Count 与真实 Broadcast Fan-out 的关系；
- 没有把 Microbenchmark 外推成整服 CCU。

26. 实验四：Core Dump 演练

在隔离环境使用专门的测试崩溃点，完成：

1. 保留匹配 ELF、Symbol、Module、Build ID；
2. 收集 Core；
3. 用 GDB 找 Signal 和 Faulting Thread；
4. 导出所有线程栈；
5. 关联崩溃前结构化日志/Trace；
6. 写出 Root Cause、影响面和 Regression Test；
7. 验证 Core 的访问和销毁策略。

不要为了演练向正常共享/生产进程发送 Crash Signal。

27. 生产变更审查模板

任何 Service 变更先回答仓库 Mandatory Review：

1. 该 Service 拥有哪些状态？
2. 所有可能 yield 的位置在哪里？
3. yield 前读取的状态在恢复后是否失效？
4. `call` 是否真的必要，`send` 是否足够？
5. 是否制造了中心代理或 Hot Actor？
6. stale fd/service/timer message 是否需要 generation/version？

再补充生产问题：

7. 新路径的 Log/Metric/Trace 是什么？
8. Timeout、Retry、Backpressure 和 Idempotency 协议是什么？
9. 如何确定性复现失败？
10. 性能预算和 Benchmark 方法是什么？
11. Config/Protocol/State Schema 如何兼容？
12. 灰度停止条件和回滚条件是什么？
13. Core/Profile 是否能关联到准确 Build？
14. Debug/管理接口是否扩大权限面？

28. 上线门禁

正确性

- Unit/Contract/Integration/Failure/Replay Test 全绿；
- 所有已发现 Bug 有 Regression Test；
- State Owner、yield、version、idempotency 已审查；
- 协议兼容和恶意输入测试完成。

可观测性

- SLO、Dashboard、Alert 和 Runbook 已存在；
- Log/Metric/Trace 可按 Build/Node/Service/Request 关联；
- Mailbox、RPC Latency、GC、Socket Buffer 可见；
- Sampling、Cardinality、Privacy 经过评审。

性能

- 真实负载模型通过；
- P99、Error Rate、CPU、Memory 满足预算；
- 发现第一饱和点和恢复行为；
- 单实例故障后仍满足降级目标；
- Soak 无持续积压或不可解释增长。

发布与恢复

- Artifact 可重建，Symbol/Core 可匹配；
- Config、Migration、Protocol 有版本；
- Canary、Stop Condition、Rollback 已演练；
- Graceful Drain 与强制退出后恢复都已验证；
- Debug Console、GDB、Dump 权限已收口。

29. 自测题

29.1 语义题

1. 为什么 Service Mailbox 为 0 仍可能有高延迟？
2. 为什么 PlayerAgent 在 `skynet.call(Scene)` 恢复后必须重新检查 version？
3. `task` 看见 coroutine 等待 Storage 时，能否证明 Storage CPU 高？
4. `mem` 降低而 RSS 不降，为什么不能立即判定内存泄漏？
5. Skynet Endless Monitor 为什么不等价于 RPC Deadlock Detector？
6. Debug Console 的 `ping` 为什么不等于玩家 RTT？
7. 为什么 `player_id` 不适合作为 Metric Label，却适合进入受控 Trace？
8. AOI QPS 提升为何可能不改善整服 P99？
9. LuaPanda 能命中业务断点，为何仍不能进入正常生产启动路径？
10. 数据库 Commit 成功但 Response 丢失时，普通 Retry 有什么风险？

29.2 设计题

1. 为 Login -> Agent -> Scene -> Storage 设计 Context 字段和传播规则。
2. 为 Scene 定义 Mailbox、Tick、AOI、Broadcast 指标及 Cardinality。
3. 为 offline/reconnect 竞态设计 Barrier-based Regression Test。
4. 为 20,000 CCU 主城热点设计负载分布和停止条件。
5. 设计 Debug Console 的生产访问控制和审计策略。
6. 给出一次 Native Crash 从告警、摘流量、Core 到修复回归的流程。
7. 说明何时把大 Scene 分区，哪些指标证明收益大于复杂度。
8. 为慢客户端写缓冲增长设计 Backpressure 与 Disconnect Policy。

29.3 实操验收

- 能从 `stat/task/trace/netstat` 形成一次同步等待链证据；
- 能用 GDB 区分 Worker、Timer、Socket、Monitor Thread；
- 能解释 `skynet_monitor.c` 的 version 检测逻辑；
- 能产出有环境、分位数、正确性和 before/after 的 Benchmark Report；
- 能列出当前仓库所有已实现与未实现的生产能力，不夸大样例成熟度。

30. 源码与命令索引

主题	入口
Worker/Timer/Socket/Monitor Thread	<code>third_party/skynet/skynet-src/skynet_start.c</code>
Endless Version 检测	<code>third_party/skynet/skynet-src/skynet_monitor.c</code>
Service STAT C 实现	<code>third_party/skynet/skynet-src/skynet_server.c</code>
Lua task/mqlen/trace API	<code>third_party/skynet/luailib/skynet.lua</code>
Debug Protocol	<code>third_party/skynet/luailib/skynet/debug.lua</code>
Debug Console 命令	<code>third_party/skynet/service/debug_console.lua</code>
开发配置与端口	<code>config/game.lua</code>
完整回归	<code>./scripts/linux/test.sh</code>
AOI Microbenchmark	<code>./scripts/linux/benchmark_aoi.sh</code>
LuaPanda/GDB 手册	<code>docs/16_DEBUGGING.md</code>
当前生产差距	<code>docs/13_PRODUCTION_GAPS.md</code>

常用诊断命令：

```
# Debug Console
./scripts/linux/debug_console.sh

# 完整测试
./scripts/linux/test.sh

# AOI 微基准
./scripts/linux/benchmark_aoi.sh 10000 100000

# Native CPU Sampling ( Staging )
perf record -F 99 -g -p <pid> -- sleep 30

# Core Analysis
gdb ./third_party/skynet/skynet /path/to/core
```

31. 四课完成后，你应具备的完整运行时地图

Build Artifact / Config Lua State

- > skynet_start
- > Worker + Global Queue + Service Mailbox
- > snlua / loader / bootstrap / main
- > Gate -> PlayerAgent -> Scene -> Storage
- > Sproto Request/Response/Push
- > coroutine yield/resume + state version
- > Log / Metric / Trace / Profile / Dump / Replay
- > Load Model / Capacity / Canary / Rollback

第一课解决“进程怎样启动、怎样断点”；第二课解决“Service 与 coroutine 到底怎样执行”；第三课解决“MMO 状态和业务闭环怎样保持一致”；第四课解决“真实故障怎样留下证据、怎样复现、怎样定量优化并安全发布”。

课程结束不意味着所有生产组件已经写完。真正的毕业标准是：面对一个未知故障或架构改动，你能从源码和运行证据提出可证伪假设，能设计确定性验证，能守住 State Ownership 与 coroutine correctness，并能用数字决定优化、扩容、分片或回滚。